Autorizador de transações Pix

Design v1.0 | 28 de setembro de 2026 | Documento anterior à implementação

1. Escopo, contrato e arquitetura

Objetivo. Autorizar transferências entre contas com saldo nunca negativo, efeito único por transação e consultas de saldo e extrato. A meta é p99 HTTP < 300 ms com 1 milhão de contas e pico de 2.000 requisições/s tocando a mesma conta, que recebe e paga. Capacidade e latência são hipóteses a validar em carga; nenhum resultado de benchmark é afirmado aqui.

Premissas. BRL em centavos inteiros, sem cheque especial, câmbio ou tarifas. A origem debitada é uma conta local; o destino pode ser local ou externo. O chamador autenticado só pode debitar contas permitidas. Para destino externo, APPROVED significa autorização e débito local persistidos, não confirmação de crédito na outra instituição. Uma integração Pix real fica fora do protótipo; um destinatário simulado, idempotente e consultável representa esse limite.

Componente	Responsabilidade e motivo
Monólito modular em Go	HTTP, validação, autorização, consultas e integração em um executável; reduz chamadas no caminho crítico e facilita testar e demonstrar em 48 h. Duas instâncias exercitam concorrência real.
PostgreSQL primário	Autoridade única para decisões, saldos e histórico. Transações ACID e constraints mantêm débito, crédito, extrato e idempotência juntos. Um volume Docker preserva dados após reinício de processo.
Outbox + despachante	Guarda intenção de envio externo no commit financeiro e permite repetir a entrega após falhas, sem chamada de rede externa dentro da transação.
Simulador externo	Persiste um efeito por ID e responde a consultas de estado; permite demonstrar timeout e reenvio sem alegar integração bancária real.
Docker Compose, k6 e auditor	Reprodução local, tráfego de chegada aberta e verificação independente das invariantes após a carga.

Caminho interno. POST /transfers com transaction_id gerado e conservado pelo cliente, origem, destino e amount_minor. A API valida formato e permissão; executa uma transação SQL; responde APPROVED ou DECLINED somente após COMMIT. GET /transfers/{id} recupera a decisão. GET /accounts/{id}/balance lê o primário. GET /accounts/{id}/statement pagina por sequência da conta; o primeiro pedido fixa a sequência máxima para as páginas seguintes.

SLO e indisponibilidade. O p99 < 300 ms refere-se a decisões conclusivas com a autoridade financeira saudável. Se o banco ficar indisponível por segundos, o serviço retorna erro técnico com prazo limitado. Se perdeu a conexão durante COMMIT, o resultado é desconhecido; repetir com o mesmo ID ou consultar. Erro técnico jamais é uma negativa de negócio. Não existe autorização baseada em saldo em cache.

2. Dados, concorrência e não duplicidade

Tabela	Campos principais e índices
accounts	id PK, kind CLIENT ou EXTERNAL_CONTROL, currency, balance_minor CHECK >= 0, last_sequence. A base tem 1 milhão de contas de clientes e 32 contas técnicas de saída, escolhidas por hash estável da origem.
operations	transaction_id PK, principal_id, payload canônico, source_id, destination_id ou destination_external, amount_minor, decision, reason, settlement_status, reverses_id UNIQUE quando presente, created_at. Registra aprovações e negativas.
entries	operation_id FK, leg (DEBIT/CREDIT), account_id FK, delta_minor, account_sequence, balance_after_minor. UNIQUE(operation_id, leg) e UNIQUE(account_id, account_sequence). Lançamentos imutáveis.
outbox	operation_id UNIQUE, external_request_id igual a transaction_id, payload, state, attempts, lease_until. Um despachante retoma eventos abandonados com a mesma identidade.

Invariantes. Uma aprovação tem exatamente duas partidas de mesmo valor e sinais opostos; negativa tem zero. Cada saldo equivale ao inicial mais a soma das partidas da conta. Todo prefixo ordenado do extrato deve ter saldo >= 0. O saldo materializado é confirmado no mesmo COMMIT das partidas. A soma dos saldos de clientes e contas técnicas permanece igual à soma inicial do domínio local, considerados os estornos. Um CHECK de linha protege saldo negativo; a igualdade entre partidas exige o protocolo de escrita e é verificada pelo auditor. [1]

Transação por transferência: versão inicial. Em READ COMMITTED, tentar inserir operations pelo transaction_id único. Se o ID já existe, comparar o payload e o principal: conteúdo igual retorna a decisão salva, conteúdo diferente retorna conflito. A restrição UNIQUE resolve duas requisições simultâneas com o mesmo ID; a perdedora espera a vencedora e relê seu resultado. Para uma nova operação, bloquear origem e destino com SELECT ... FOR NO KEY UPDATE em ordem crescente de ID, uma linha por vez. Validar moeda, existência, tipo de conta, valor positivo e ausência de overflow. Debitar apenas se houver saldo suficiente; creditar destino; inserir duas partidas e decisão. Para saldo insuficiente, gravar DECLINED sem partidas. COMMIT precede a resposta. Créditos e débitos de uma mesma conta disputam sua linha, preservando uma ordem real; contas independentes seguem em paralelo. [2]

Defesa em profundidade. Na escrita do débito, condicionar UPDATE a balance_minor >= amount_minor e verificar que uma linha foi alterada. CHECK(balance_minor >= 0), índices únicos e rollback impedem estado parcial. Toda movimentação financeira, inclusive seed controlado e estorno, passa pelo mesmo protocolo. Os papéis da API não recebem permissão de UPDATE/DELETE direto nas partidas. A ordem das decisões é a ordem efetiva dos locks e commits, sem promessa de FIFO pela chegada à rede.

Idempotência. O cliente reutiliza transaction_id após timeout, inclusive se não sabe se houve commit. Uma negativa por saldo insuficiente é terminal para aquele ID; novo pedido requer novo ID. A chave fica persistida, sem depender de TTL em cache. Se a aplicação morrer antes do commit, não há efeito nem resultado terminal; se morrer depois, ambos já existem. A consulta de um ID ausente durante uma tentativa ainda em andamento não prova que ela não concluirá, portanto o cliente deve repetir o mesmo ID.

Externo. Na autorização, a origem local é debitada e uma das contas técnicas não gastáveis recebe o mesmo valor, junto de entries, operations APPROVED/EXTERNAL_PENDING e outbox. O shard técnico evita que todos os pagamentos externos disputem uma única linha. Confirmação externa muda settlement_status para SETTLED; rejeição definitiva gera uma única operação de estorno ligada à original. SETTLED e REVERSED disputam condicionalmente a mesma linha em operations; transição para REVERSED, partidas e saldos fazem um COMMIT sob o mesmo bloqueio de contas. Timeout externo mantém PENDING até reconciliação por ID. O controle técnico representa saídas locais autorizadas líquidas de estornos; não é saldo real do destinatário.

3. Hot accounts, metas e falhas

Por que a conta quente é difícil. Com 2.000 transferências/s envolvendo uma conta, cada transação que altera seu saldo compete pelo mesmo registro, inclusive créditos. Instâncias adicionais da API não removem essa disputa. A versão inicial usa transações curtas, índices por PK, pool de conexões limitado e nenhuma chamada externa durante a transação. A conta técnica externa é distribuída em 32 linhas, mas isso não resolve a disputa da origem quente. A capacidade real dessa abordagem precisa ser medida.

Decisão condicionada por benchmark. Primeiro executar a versão por transferência e registrar p99, vazão e espera por lock sob 2.000 req/s na mesma conta. Se falhar ou ficar sem margem de capacidade, implementar microbatch como otimização localizada no módulo financeiro: agrupar no máximo 64 solicitações por até 2 ms, registrar ordem do lote, validar cada débito em sequência contra saldos intermediários e gravar uma vez o saldo final de cada conta, com decisões e partidas individuais no mesmo commit. Um lock transacional do banco impede dois executores do lote simultâneos. A mudança de modo exige drenar os escritores anteriores; os dois modos não operam simultaneamente. O lote usa prazo próprio; o timeout de um cliente não cancela os demais. Essa alternativa pode elevar vazão da conta quente, mas serializa lotes de contas independentes; só entra se a medição justificar. Comparar lote 1, 16 e 64 antes de fechar a implementação. [2]

Falha exigida	Comportamento e verificação
Instância morre antes do COMMIT	PostgreSQL faz rollback; bloqueios são liberados após encerrar a conexão. Cliente repete o ID. O teste mata uma das duas instâncias sob carga.
Instância morre após o COMMIT, antes da resposta	Operação, saldo e partidas existem juntos. Reenvio devolve decisão anterior, sem nova movimentação.
Banco fica lento por segundos	Timeouts de conexão, lock e transação limitam a espera; resposta técnica. Se COMMIT foi enviado e resposta se perdeu, o resultado é desconhecido e deve ser reconciliado pelo mesmo ID.
Banco indisponível por segundos	Não aprovar nem registrar negativa financeira em memória. Responder erro técnico, preservar pedidos já confirmados e aceitar retries quando voltar.
Cliente dá timeout e reenvia	O mesmo transaction_id produz o resultado original ou uma única execução se a primeira abortou. Payload diferente com o mesmo ID gera conflito.
Destinatário externo dá timeout ou despachante cai	Outbox é retomada; o simulador reconhece o mesmo ID. Consultar estado antes de estornar; timeout isolado nunca libera fundos.

Limites da garantia. O Compose com um banco e volume demonstra recuperação de processo, não perda de disco nem failover sem perda. Em produção seria necessária uma política de backup, réplica síncrona se RPO zero for exigido e proteção contra dois primários. Preservar fsync e synchronous_commit. Nenhuma arquitetura consegue dar decisão financeira conclusiva durante segundos de indisponibilidade do único banco autoritativo e ainda cumprir p99 < 300 ms; essas falhas serão medidas e reportadas separadamente. [3]

4. Teste, trade-offs e plano

Ensaio de carga. Pré-carregar 1 milhão de contas, registrar hardware, versões e commit. Aquecer e rodar 10 min com 2.000 req/s que tocam a mesma conta, em cenários separados de fan-in, fan-out e mistura de entradas/saídas. Manter fundos suficientes no ensaio de vazão e acrescentar 200 req/s entre contas frias. Exercitar consultas concorrentes e pagamentos externos em outro ensaio. k6 usa taxa de chegada aberta; registrar taxa oferecida, iniciada, concluída e dropped_iterations, além de p50/p95/p99 HTTP ponta a ponta, espera por lock, timeouts, negativas e erros técnicos. O cenário saudável só passa com p99 < 300 ms, zero erros técnicos e sem descarte de iterações; relatório informa também vazão sustentada e margem antes da saturação. [4]

Auditoria independente. Gerar manifesto de IDs e payloads; após a carga, reconciliar respostas desconhecidas repetindo os mesmos IDs e drenar a outbox do simulador. Recalcular cada saldo e cada prefixo do extrato a partir do seed e de entries. Conferir saldo >= 0 em todo prefixo, soma dos saldos, duas partidas opostas por aprovação, zero por negativa, no máximo um movimento inicial por ID, no máximo um estorno por original e resultado conhecido para todo ID enviado. Rodar 100 saques distintos de R$ 10 contra R$ 100: exatamente 10 APPROVED, 90 DECLINED e saldo zero; repetir as 100 chaves sem novo efeito. Injetar kill -9, lentidão do banco e perda de resposta após COMMIT.

Trade-offs. Uma transação por transferência é simples de auditar e paraleliza contas independentes, mas a conta quente pode exceder o SLO por contenção. Microbatch reduz escritas e commits da conta quente, à custa de espera de agregação e serialização mais ampla; sua adoção depende do teste. PostgreSQL único simplifica atomicidade e extrato, mas concentra dependência de disponibilidade. O envio externo usa outbox e compensação; não há commit atômico entre instituições. O simulador permite provar idempotência do destinatário, não a disponibilidade ou semântica de um provedor real. Shards técnicos evitam um gargalo artificial, adicionando apenas a escolha determinística da conta técnica.

Validações pendentes. Demonstrar p99 com 2.000 req/s no hardware da entrega; confirmar que a carga adicional não sofre starvation; escolher número de instâncias, pool e quantidade de shards técnicos por medição; testar custo do extrato de conta quente e crescimento de operations/entries; medir tempo de recuperação e backlog da outbox após falhas. Se o baseline não atingir p99, documentar números, aplicar a otimização de lote e comparar sem esconder falhas. A meta de 48 h exige fechar esse ciclo cedo.

Ordem de entrega. Este design deve ser enviado aos dois avaliadores antes de iniciar o código e em até 6 h do recebimento do desafio. Depois: implementar núcleo, medir baseline, decidir sobre lote, completar consultas e simulador, executar carga e auditoria, preservar commits e registrar prompts/sessões de IA no repositório. O README trará comandos e resultados numéricos reais. Esta versão substitui a v0.2: inicia com transações por transferência e mantém microlotes como hipótese testável. Nenhum benchmark foi executado para este documento.

Referências. [1] https://www.postgresql.org/docs/current/ddl-constraints.html ; [2] https://www.postgresql.org/docs/current/explicit-locking.html ; [3] https://www.postgresql.org/docs/current/wal-async-commit.html ; [4] https://grafana.com/docs/k6/latest/using-k6/scenarios/executors/constant-arrival-rate/